King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 08

AVL-Trees

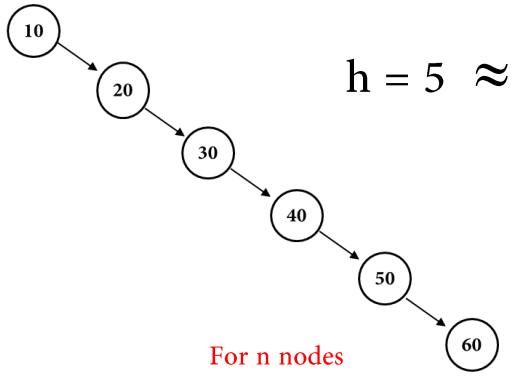
Term 261

Objectives

- Discuss the following topics:
- AVL Trees

Motivation

- Keys: 10, 20, 30, 40, 50, 60

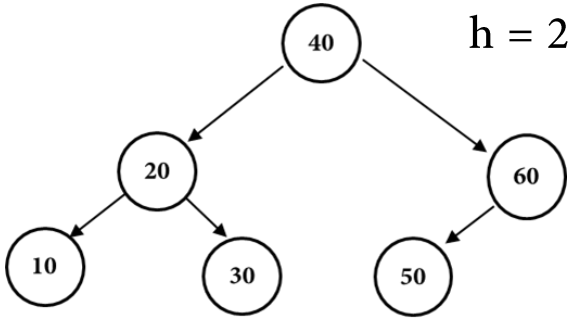


h = 5 ≈ n

For n nodes

Search takes O(n)

keys: 40, 20, 30, 60, 50, 10



h = 2 ≈ log(n)

For n nodes

Search takes O(log(n))

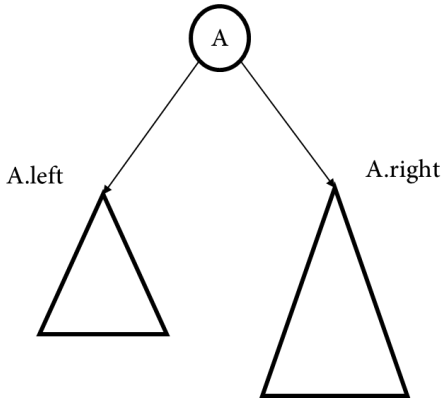
- All BST operations are O(h), where h is tree height.
  - In the best case searching a balanced BST is O(log n), shallow BST.
  - In the worst case searching an imbalanced BST is O(n), deep BST.

- Insertion or deletion in an ordinary Binary Search Tree (BST) can cause the tree to become unbalanced, increasing its height and potentially making operations less efficient.

- Solution: balance the tree, e.g. AVL tree, Splay tree

Height of a tree and Balance factor

- Height is max number of edges in a path from node to any leaf.
  - height(empty tree) = -1
  - height(a leaf) = ?
  - height(A) = ?



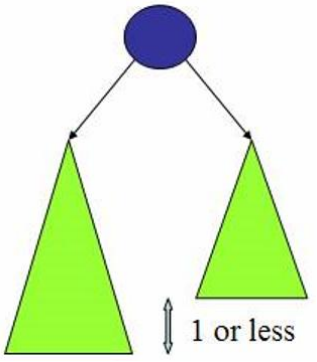
- Hint: it's recursive!
  - height(a leaf) = 0
  - height(A) = 1 + max(height(A.left), height(A.right))

- Balance factor
  - Balance factor = height(rightSubtree) - height(leftSubtree)

What is an AVL Tree?

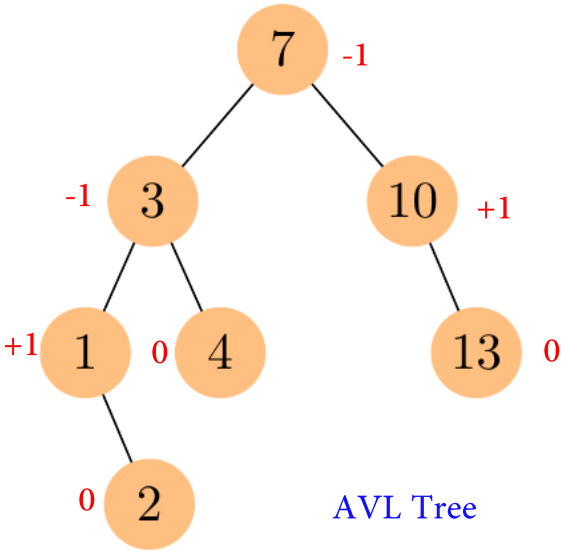
- An AVL (Adelson Velsky and Landis) is height-balanced BST
  - For each node v, the heights of its children differ by at most 1.
  - An AVL node can have a balance factor of -1, 0, or +1.
    - Balance factor = height(rightSubtree) - height(leftSubtree)

- Idea: After insertion or deletion, an AVL tree may need rebalancing to maintain its height-balance property.
  - The height-balance property ensures that the height of an AVL tree with n nodes is O(log n).
  - Searching, insertion, and deletion are all O(log n).
  - Rebalancing operations are relatively efficient.

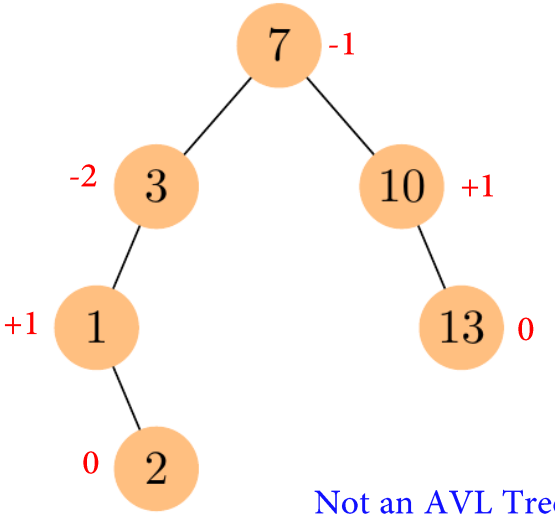


- A subtree of an AVL tree is also an AVL tree.

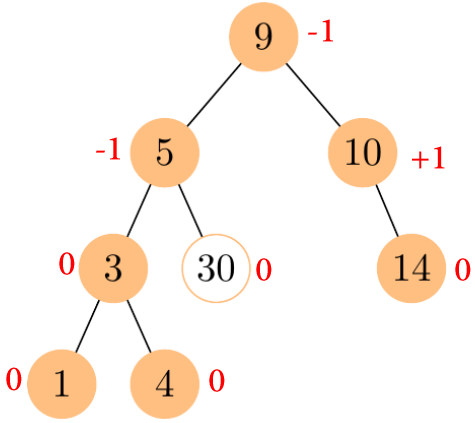
Which of the following is an AVL Tree?



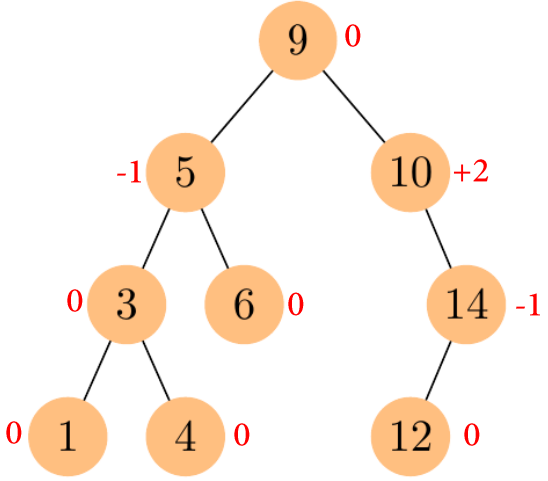
AVL Tree



Not an AVL Tree

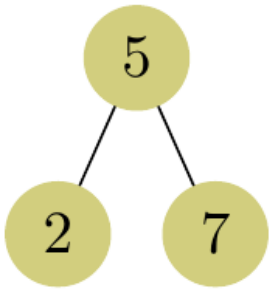


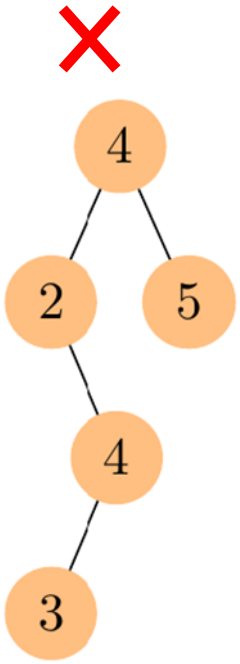
Not BST -> Not an AVL Tree

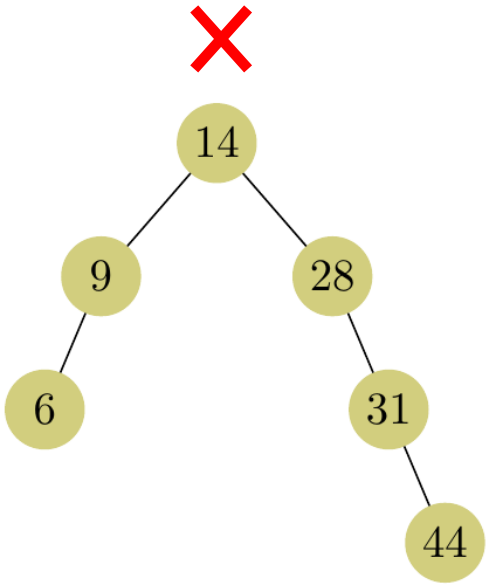


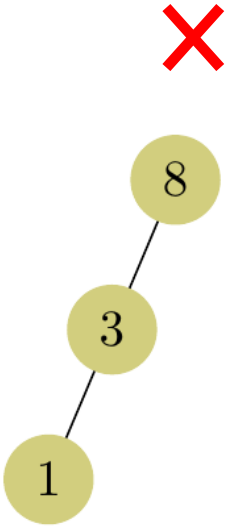
Not an AVL Tree

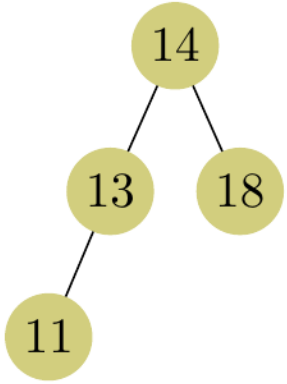
Which of the following is an AVL Tree?

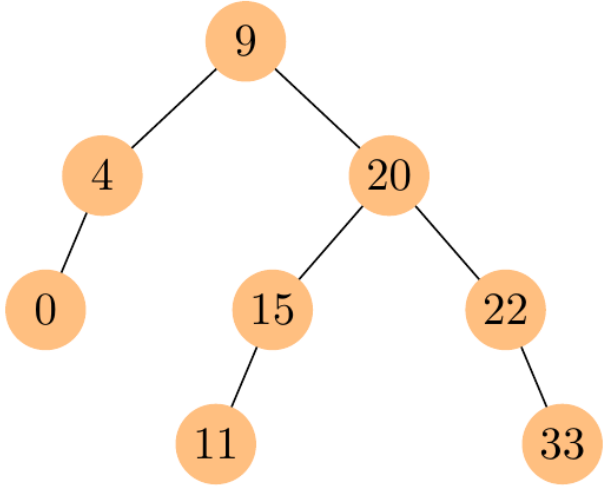












AVL Tree Implementation

```
public class AVLTree<T extends Comparable<T>> extends BST<T> {
        protected int height;
        public AVLTree() {
                super();
                height = -1;      // empty tree
        }

        public AVLTree(BTNode<T> root) {
                super(root);
                height = 0;          // A tree with one node
        }

        public int getHeight() {
                return getHeight(root);
        }

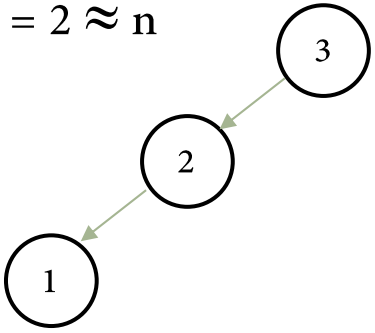
        private int getHeight(BTNode<T> node) {
                if(node == null)
                        return -1;
                else
                        return 1 + Math.max(getHeight(node.left), getHeight(node.right));
        }
         // . . .
 }
```

- Many of the AVL tree operations depend on height
- Instead of computing Height recursively, each node can keep track of its subtree height as a field

AVL Rotation operations

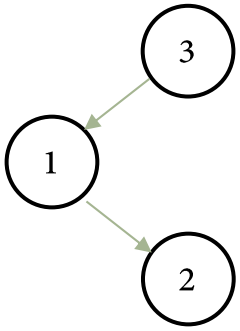
- All possible ways to insert 1, 2, 3

h = 2 ≈ n



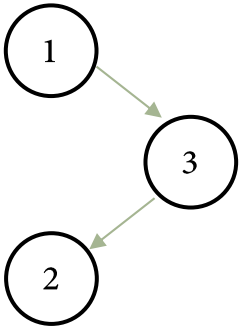
3, 2 ,1

Case 1: LL imbalance



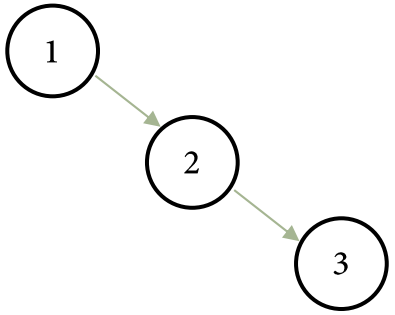
3, 1, 2

Case 2: LR imbalance



1, 3, 2

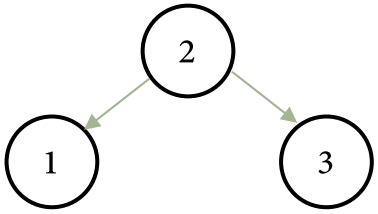
Case 3: RL imbalance



1, 2, 3

Case 4: RR imbalance

h = 1 ≈ log(n)



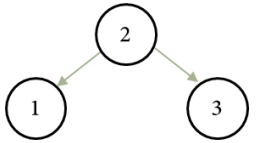
2, 1, 3

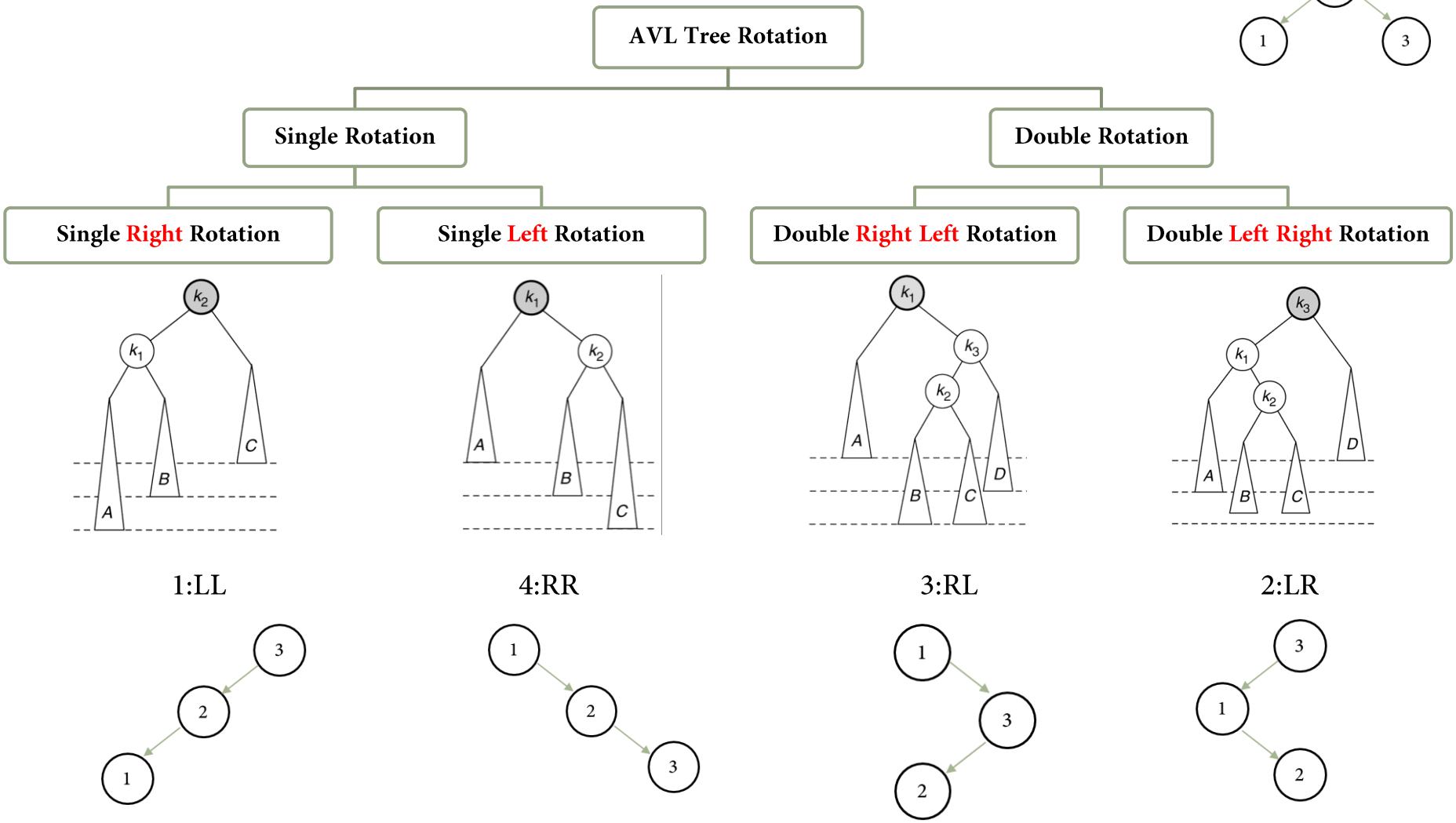
2, 3, 1

1. Insertion in the left subtree of the left child of a node.
2. Insertion in the left subtree of the right child of a nod.
3. Insertion in the right subtree of the left child of a node.
4. Insertion in the right subtree of the right child of a node.

What is a Rotation?...

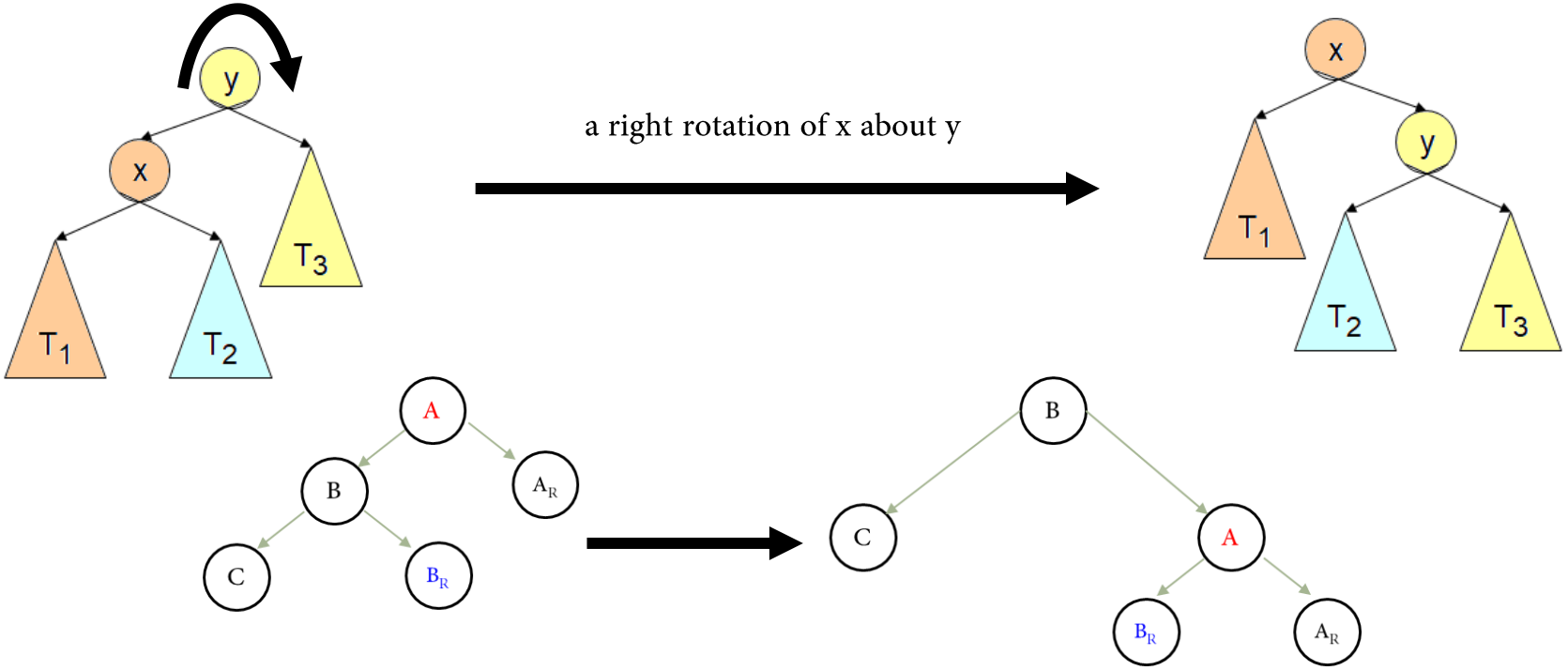
- A rotation is a process of switching children and parents among two or three adjacent nodes to restore balance to a tree.





BST ordering property after a rotation

- A rotation does not affect the ordering property of a BST.



BST ordering property requirement:

$T_1 < x < y$

$x < T_2 < y$

$x < y < T_3$

Similar

BST ordering property requirement:

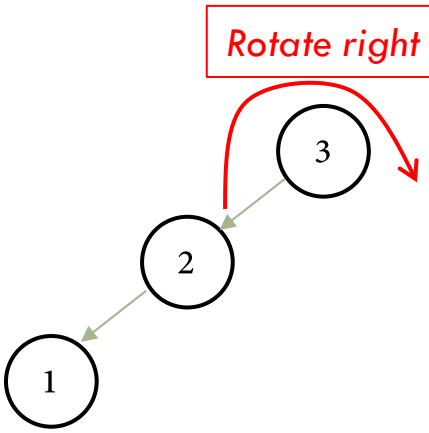
$T_1 < x < y$

$x < T_2 < y$

$x < y < T_3$

- Similarly for a left rotation.

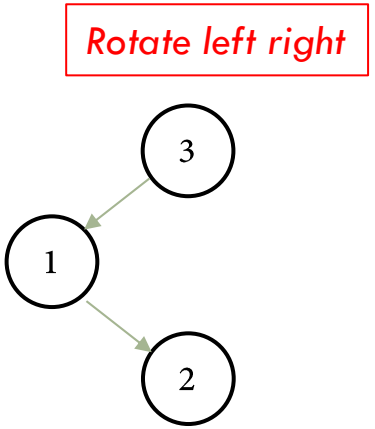
AVL Rotation operations…



Case 1: LL imbalance

3 becomes the right child of 2

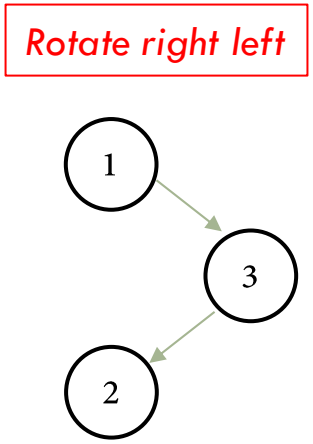
2 becomes the root



Case 2: LR imbalance

2 becomes the root

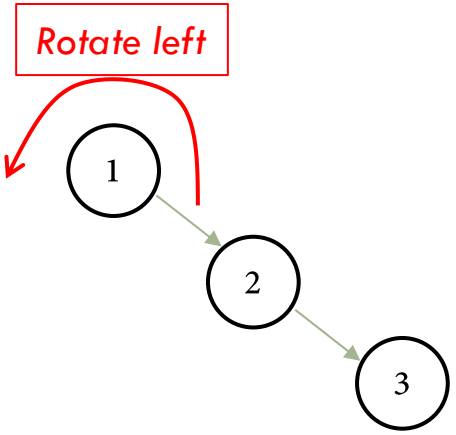
3 becomes the right child of 2



Case 3: RL imbalance

2 becomes the root

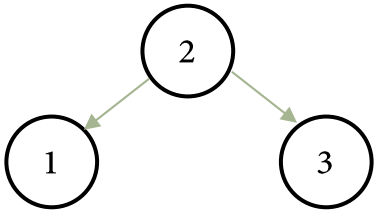
1 becomes the left child of 2



Case 4: RR imbalance

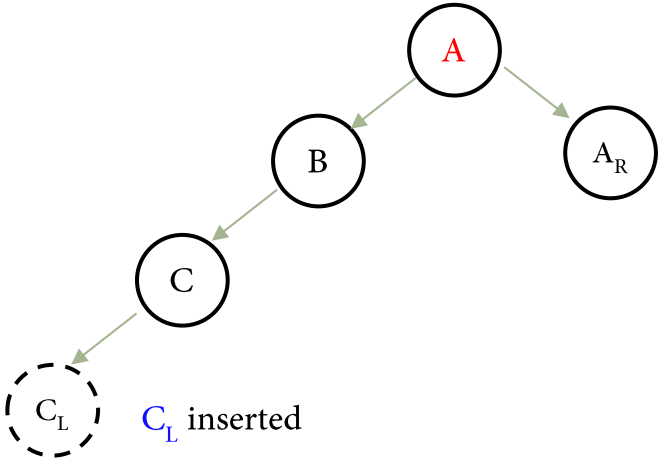
1 becomes the left child of 2

2 becomes the root



AVL Rotation operations…

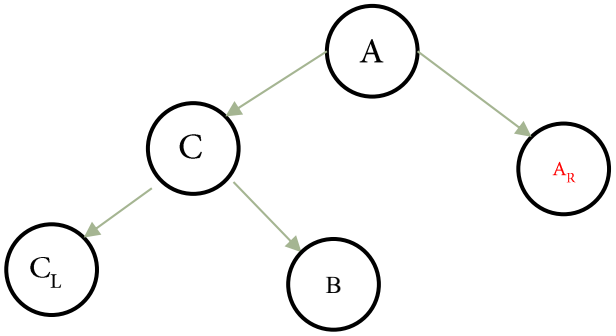
Case 1: LL- *Rotate right*

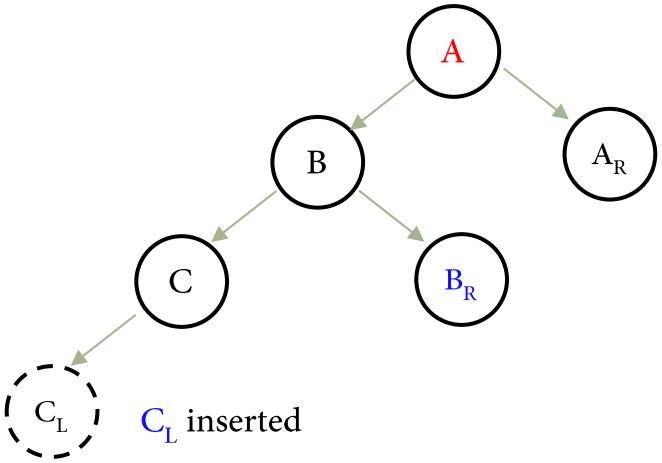


C_L inserted

B becomes the right child of C

C becomes left child of A

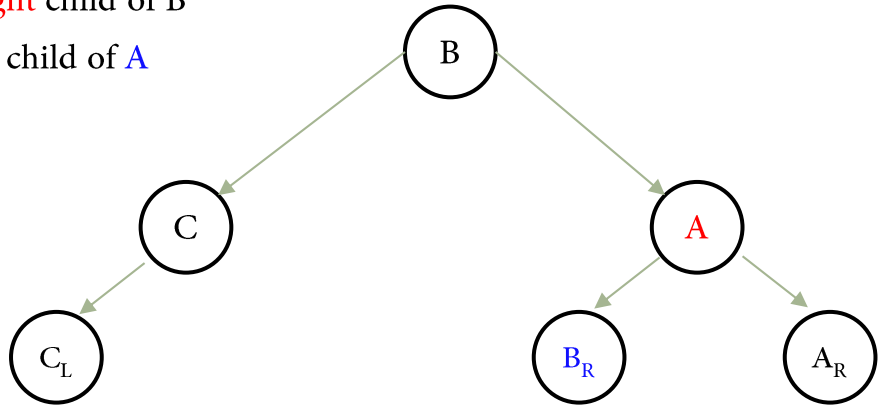




C_L inserted

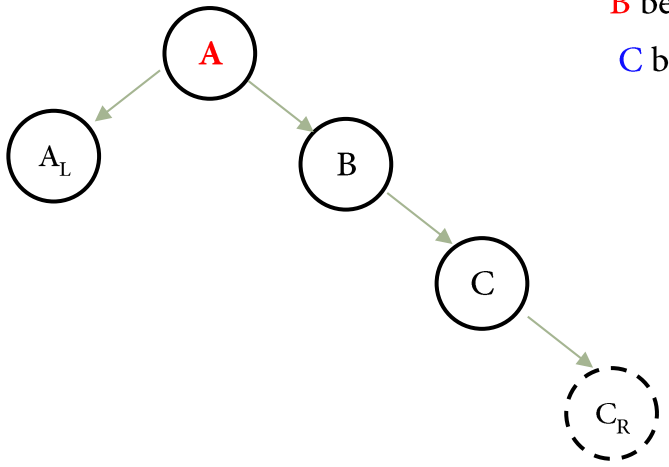
A becomes the right child of B

B_R becomes left child of A



AVL Rotation operations…

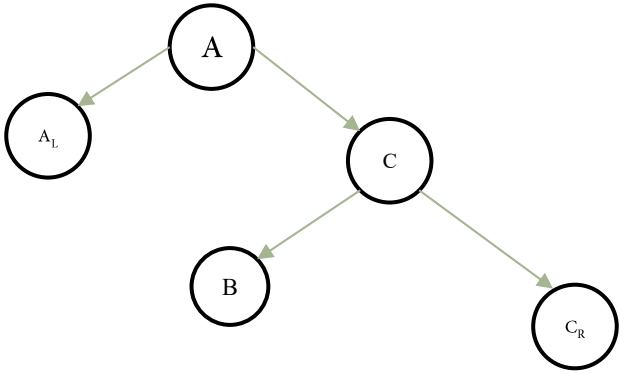
Case 2: RR- *Rotate left*

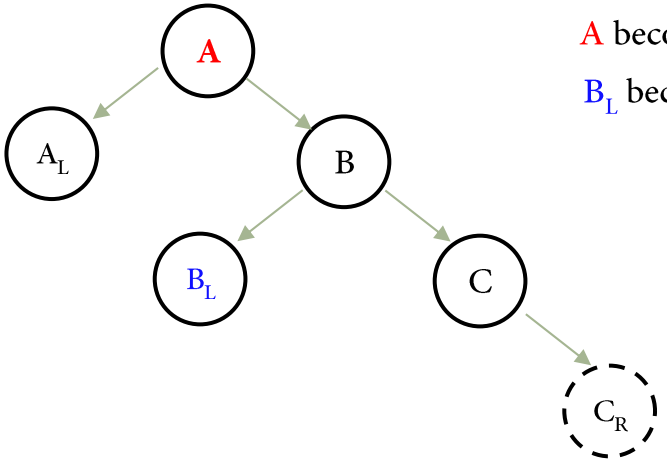


C_R inserted

B becomes the left child of C

C becomes right child of A

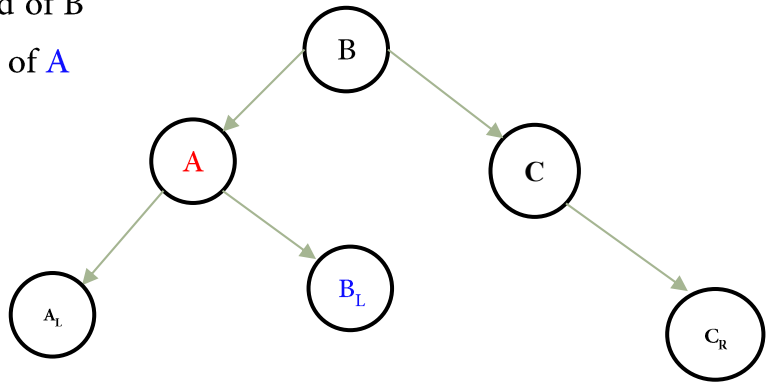




C_R inserted

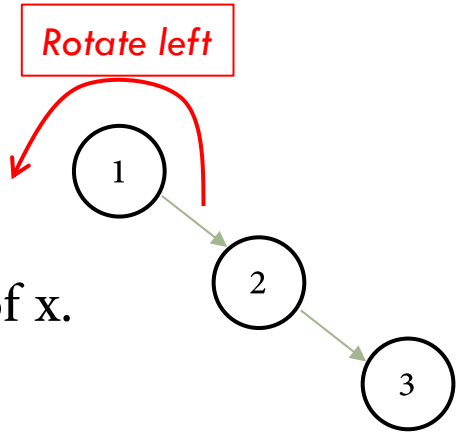
A becomes the left child of B

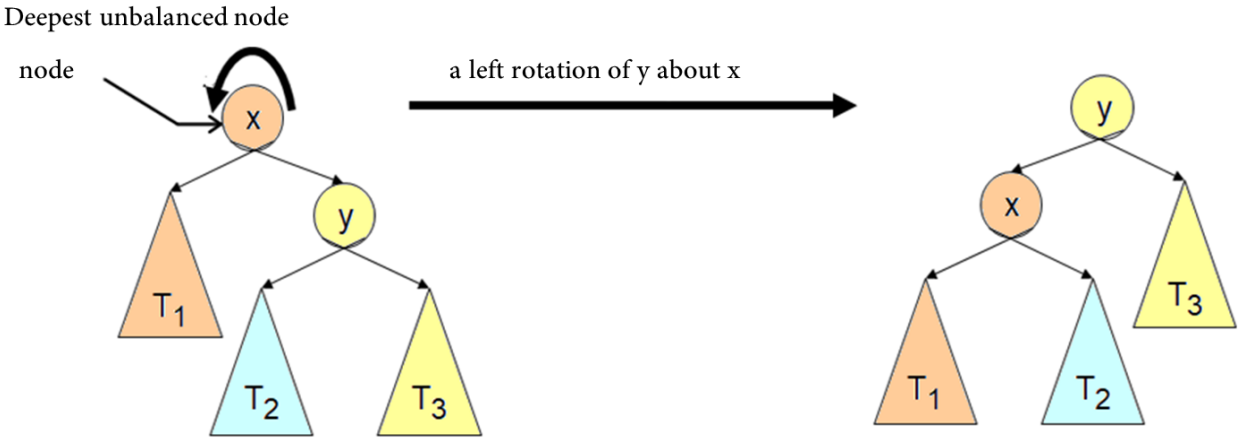
B_L becomes right child of A



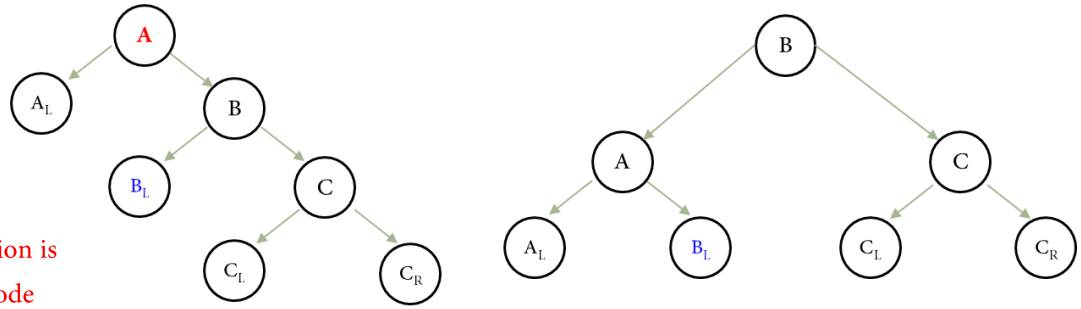
Single Left Rotation (RR)

- Single left rotation:
  - The right child y of a node x becomes x's parent.
  - x becomes the left child of y.
  - The left child T2 of y, if any, becomes the right child of x.





RR



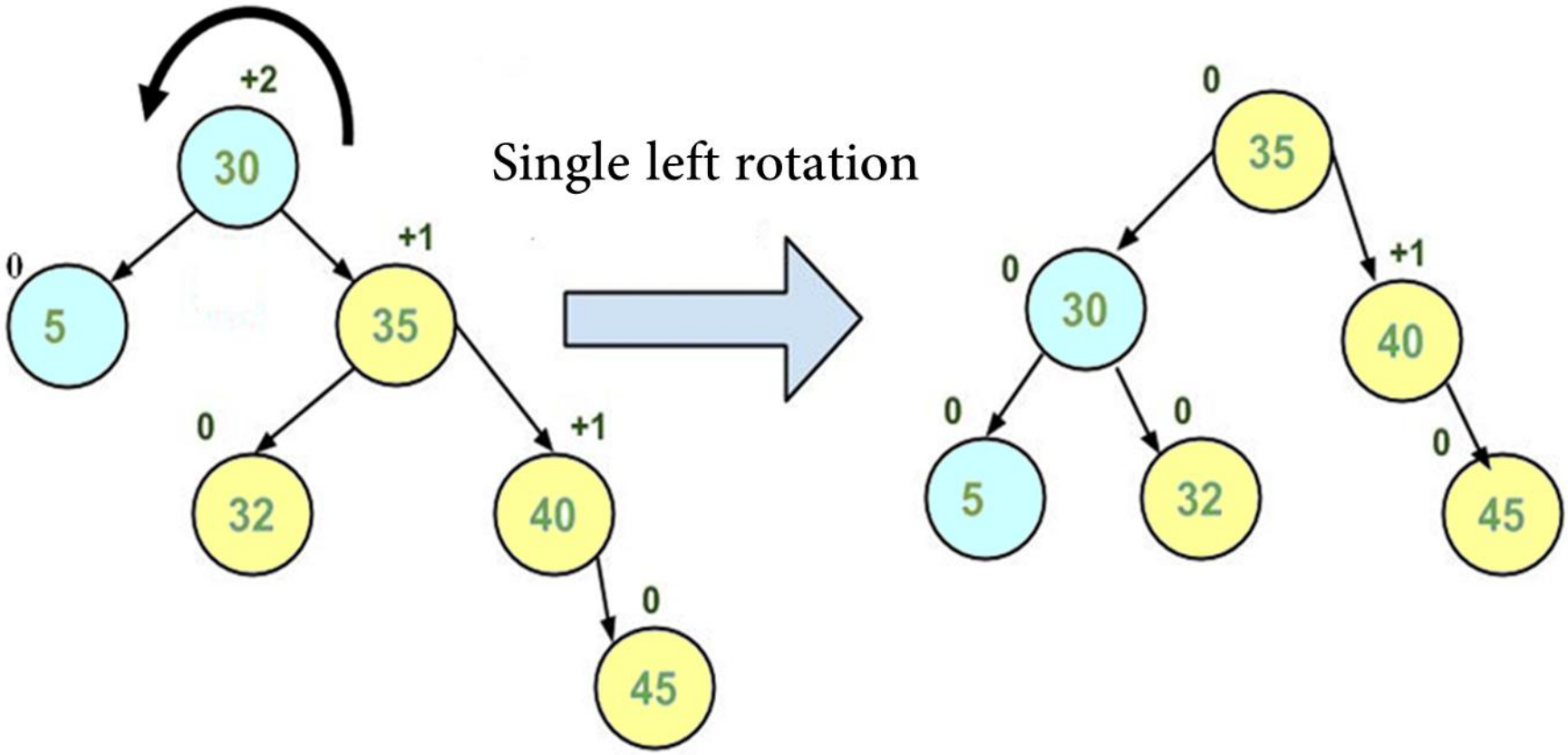
Note: The pivot of the rotation is the deepest unbalanced node

A becomes the left child of B

B becomes the root

B_L becomes right child of A

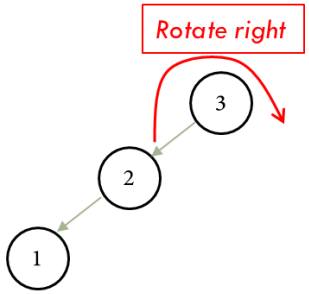
Single Left Rotation Example

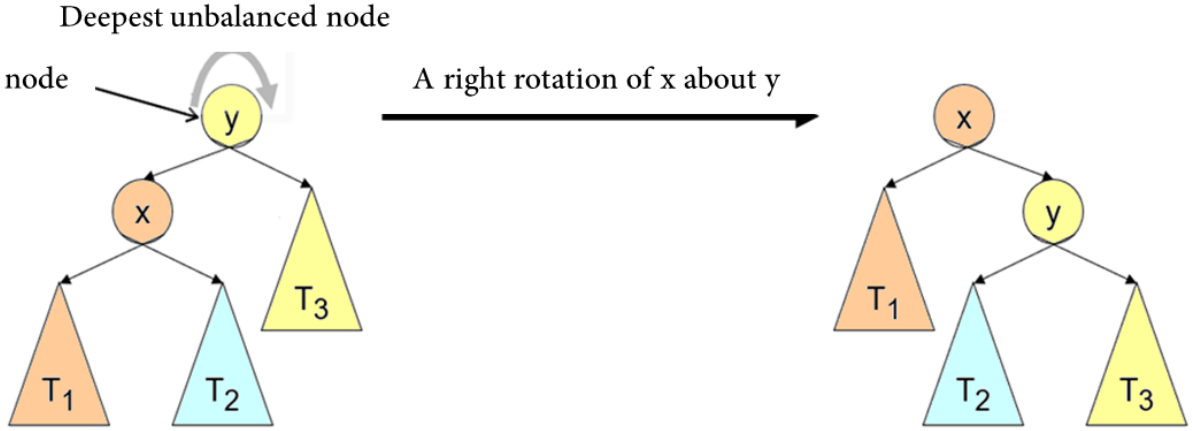


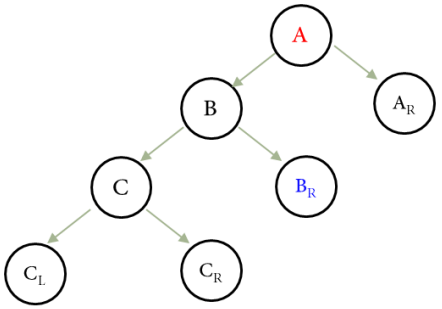
Left rotate at node 30, producing a tree rooted at 35 with 30 as left child OF 35 and 32 as right child OF 30.

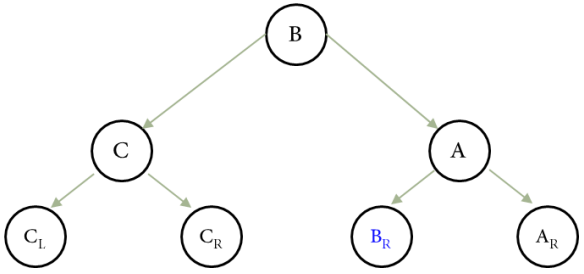
Single Right Rotation (LL)

- Single right rotation:
  - The left child x of a node y becomes y's parent.
  - y becomes the right child of x.
  - The right child T2 of x, if any, becomes the left child of y.









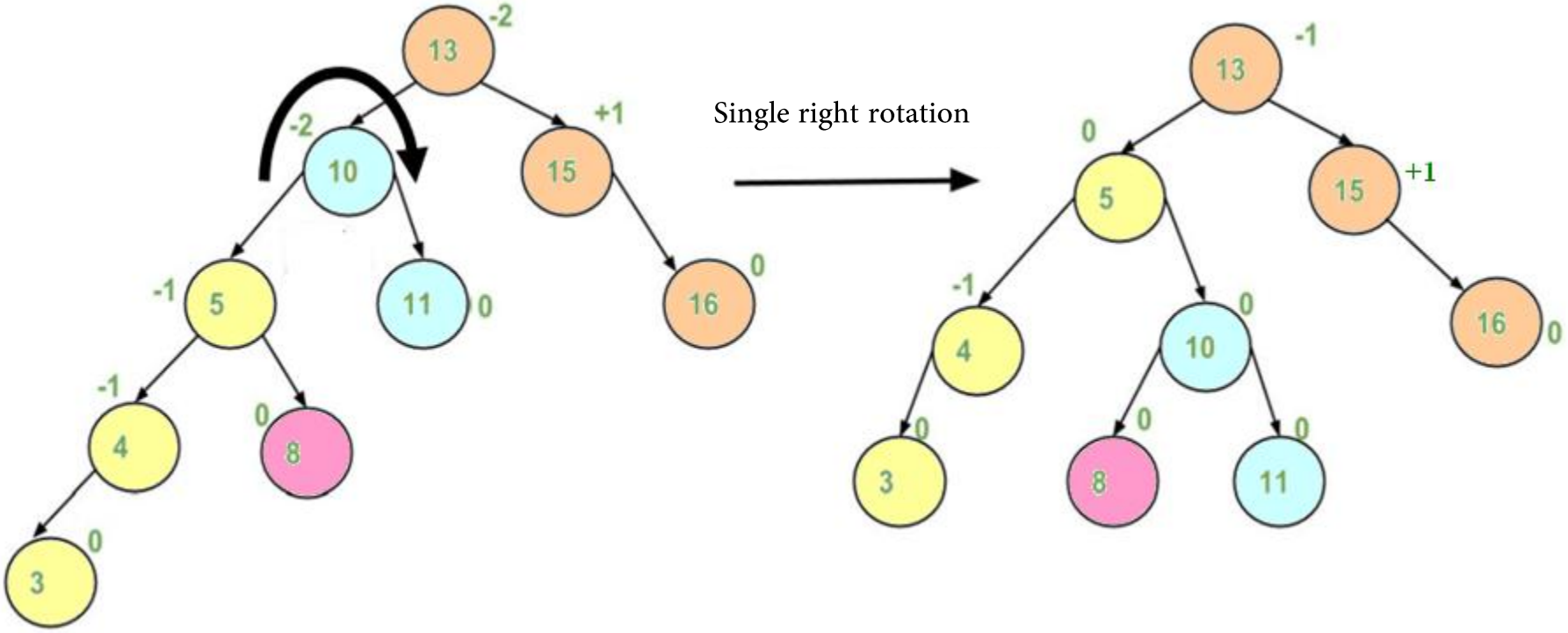
Note: The pivot of the rotation is the deepest unbalanced node

A becomes the right child of B

B becomes the root

B_R becomes left child of A

Single Right Rotation (Example1)

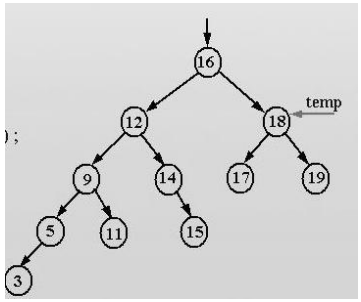


Right rotate at node 10, producing a left subtree rooted at 5 with 10 as right child OF 5 and 6 as right child OF 10.

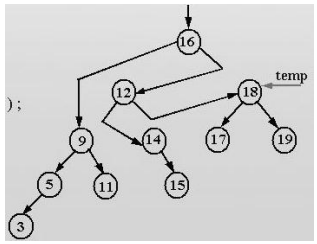
Single Right Rotation Implementation (example)

```
1  protected void RotateRight(){
2  If ( isEmpty()) throw new Exception()
3        BSTNode<T> tempNode = root.right;
4        root.right = root.left;
5        root.left = root.right.left;
6        root.right.left= root.right.right;
7        root.right.right = temp;
8        Object tmpObj = key;
9        root.key = root.right.key;
10       root.right.key = tmpObj;
11       getRigthAVL().adjustHeight();
12       adjustHeight();
13 }
```

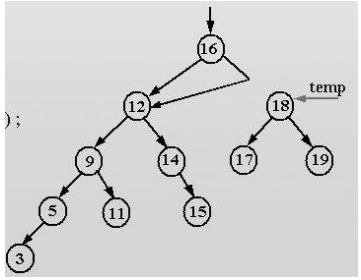
3



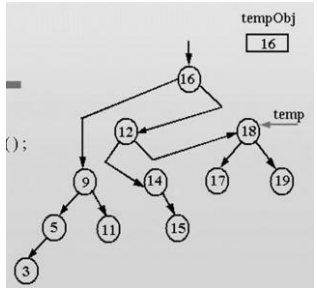
7



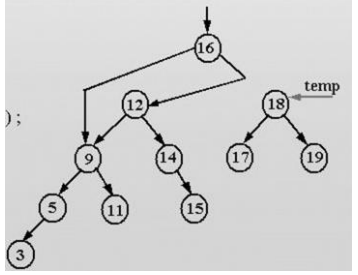
4



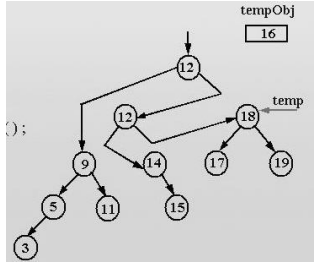
8



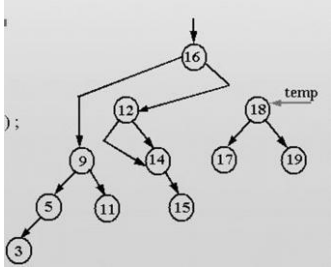
5



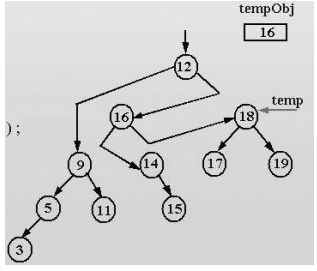
9



6



10



AVL Rotation operation…

Case 3: LR- *Rotate left right*

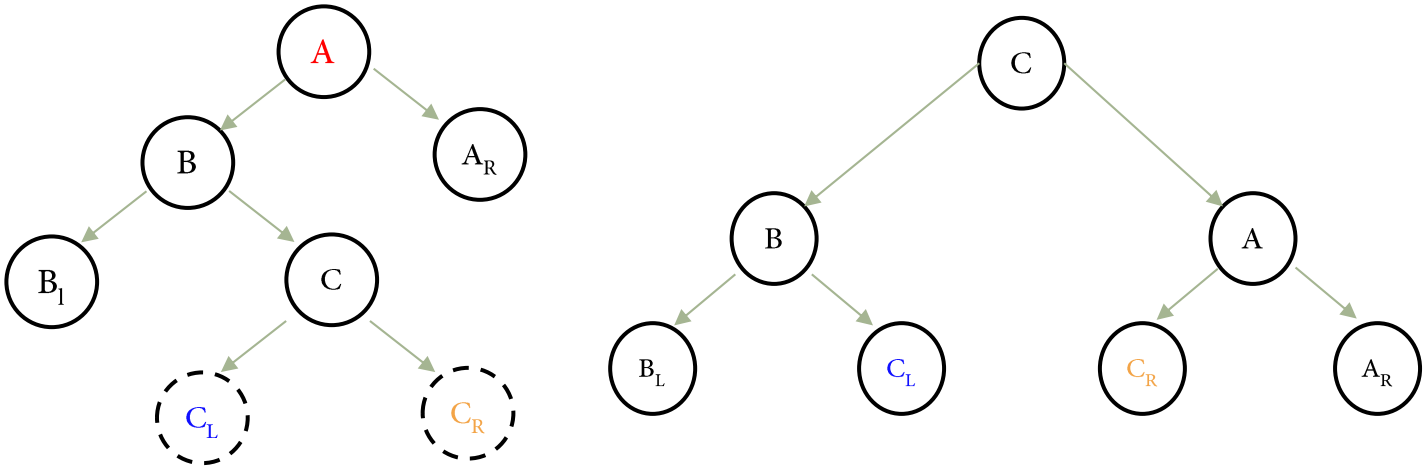
C replaces A

A becomes the right child of C

B becomes the left child of C

C_L becomes the right child of B

C_R becomes the left child of A



Case 4: RL- *Rotate right left*

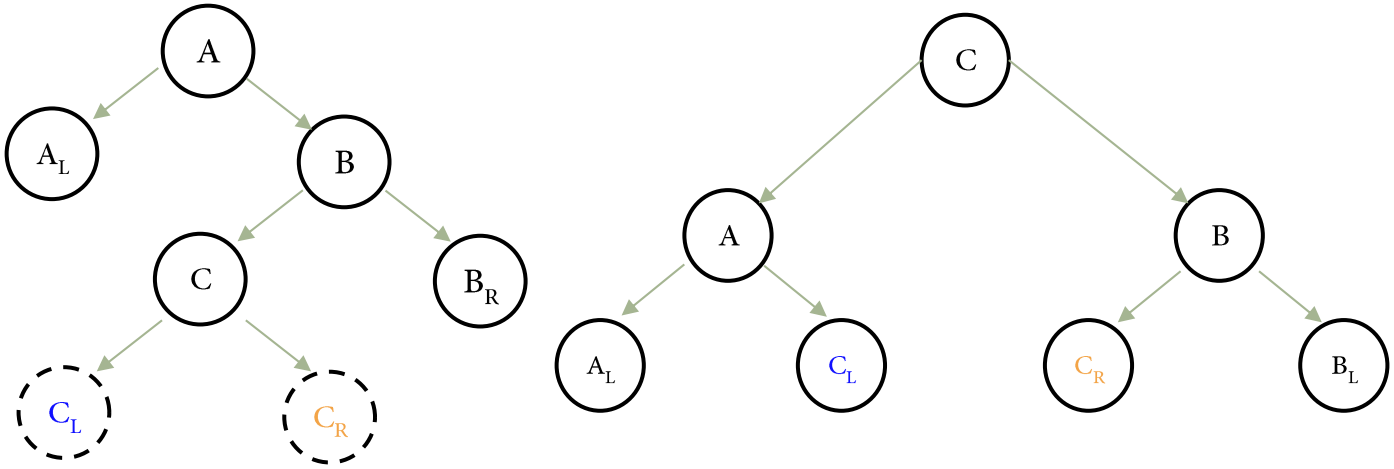
C replaces the root

A becomes the left child of C

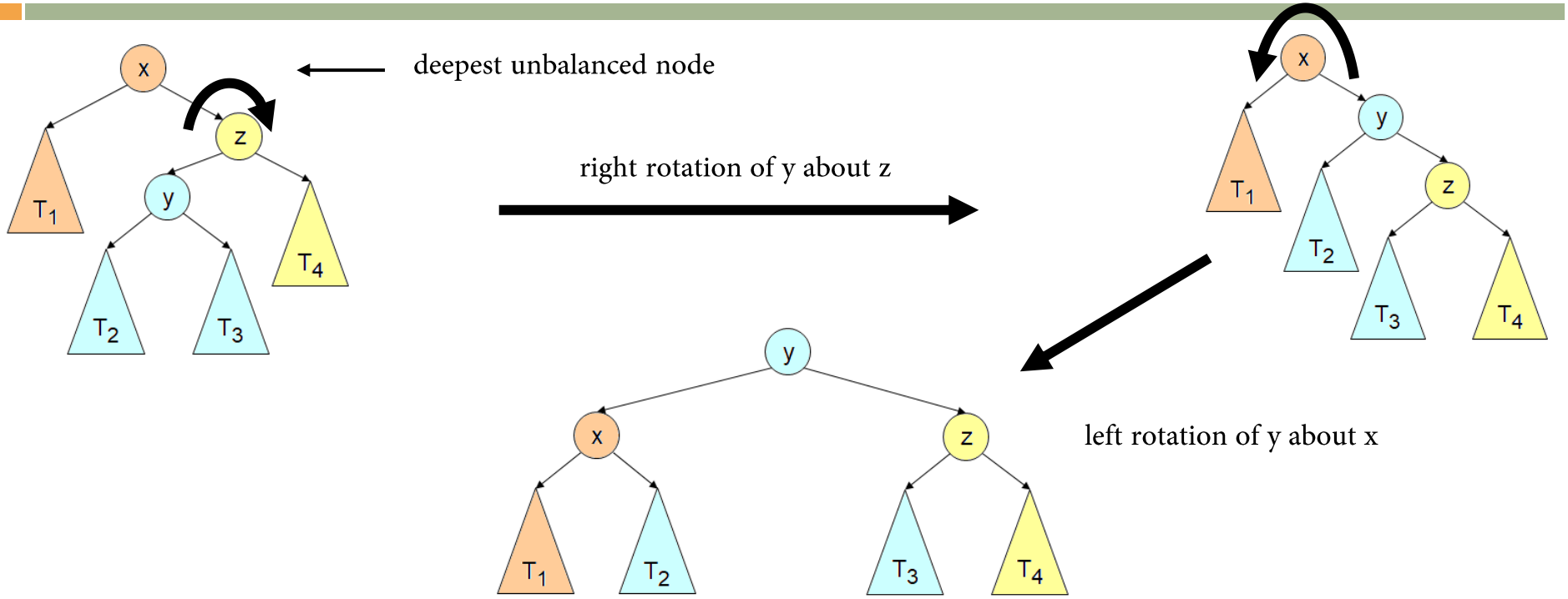
B becomes the right child of C

C_L becomes the right child of A

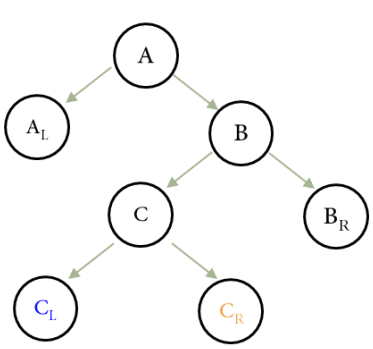
C_R becomes the left child of B

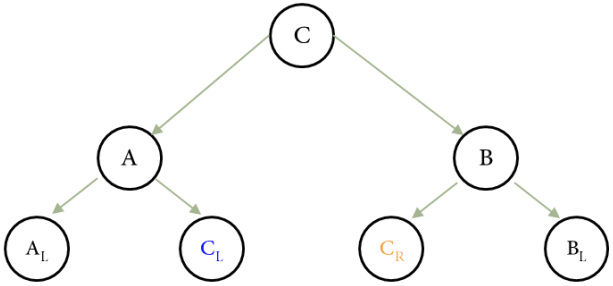


Double Right-Left Rotation



RL





C becomes the root

A becomes the left child of C

B becomes the right child of C

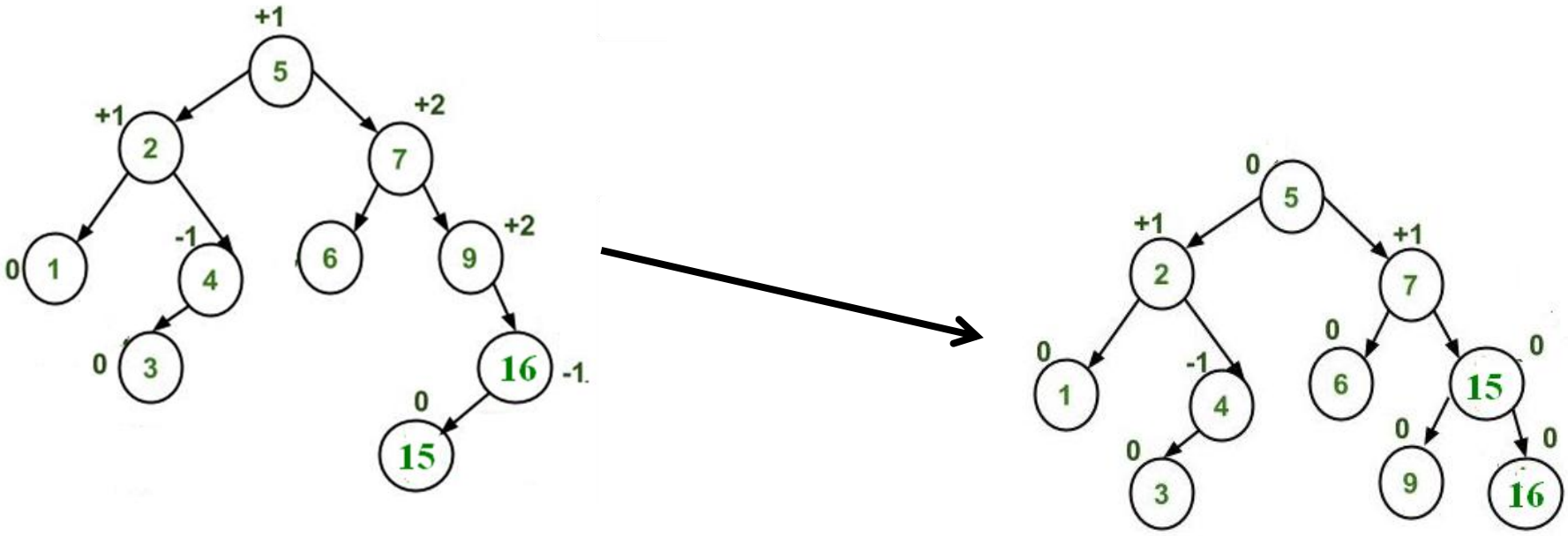
C_L becomes the right child of A

C_R becomes the left child of B

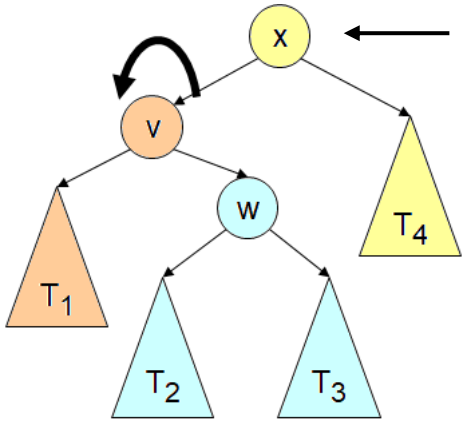
Note: First pivot is the right child of the deepest unbalanced node; second pivot is the deepest unbalanced node

Double Right-Left Rotation Example

- 9 is imbalanced. It is RL case. For RL, consider 3 nodes 9, 16 and 15
- 15 moved up, it replaces 9
- 9 becomes left child of 15

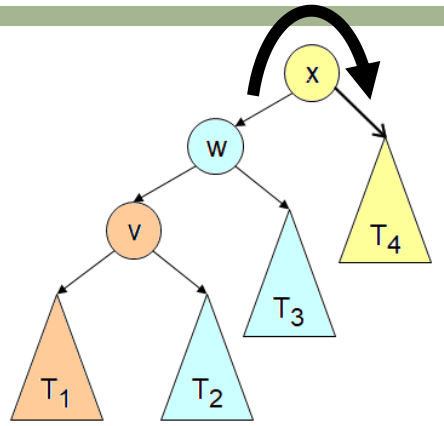


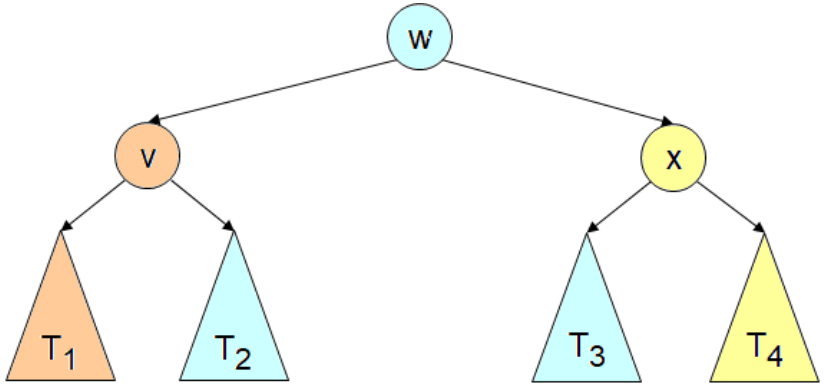
Double Left-Right Rotation



deepest unbalanced node

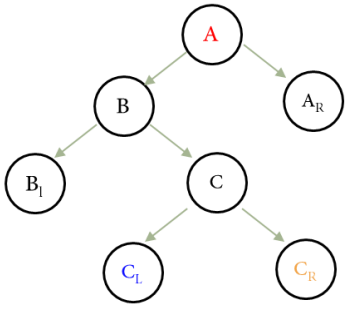
left rotation of w about v

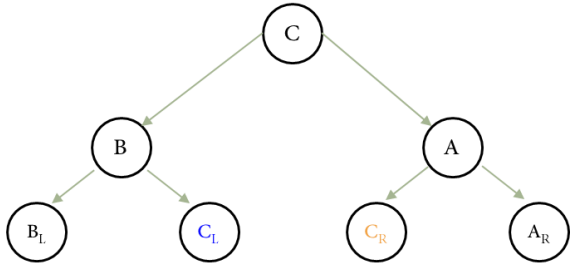




left rotation of W about X

LR





C becomes the root

A becomes the right child of C

B becomes the left child of C

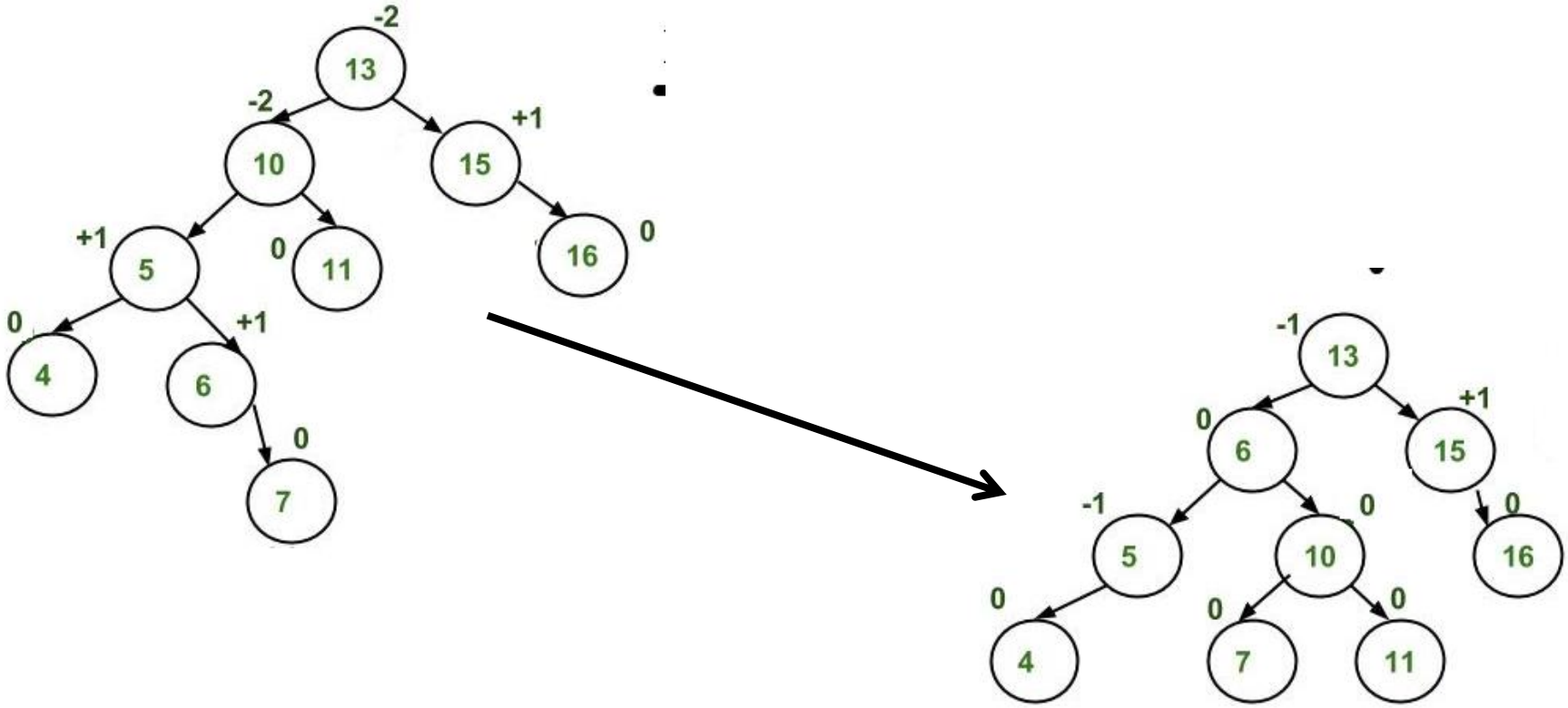
C_L becomes the right child of B

C_R becomes the left child of A

Note: First pivot is the left child of the deepest unbalanced node; second pivot is the deepest unbalanced node

Double Left-Right Rotation Example

- 10 is imbalanced. It is LR case. For LR, consider 3 nodes 10, 5 and 6
- 6 moved up, it replaces 10
- 10 becomes right child of 6
- 7 becomes left child of 10

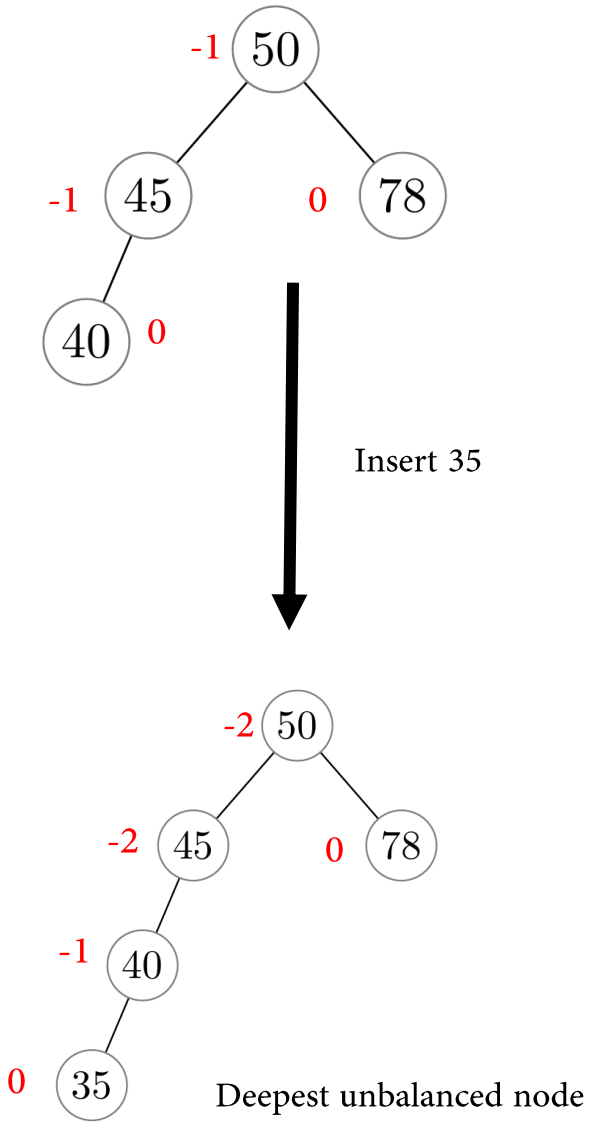


AVL Insertion

1. Perform standard BST insertion.
2. Update the heights.
3. Calculate the balance factor.
4. Find the first unbalanced node.
5. Identify LL, RR, LR, or RL.
6. Perform the appropriate rotation.

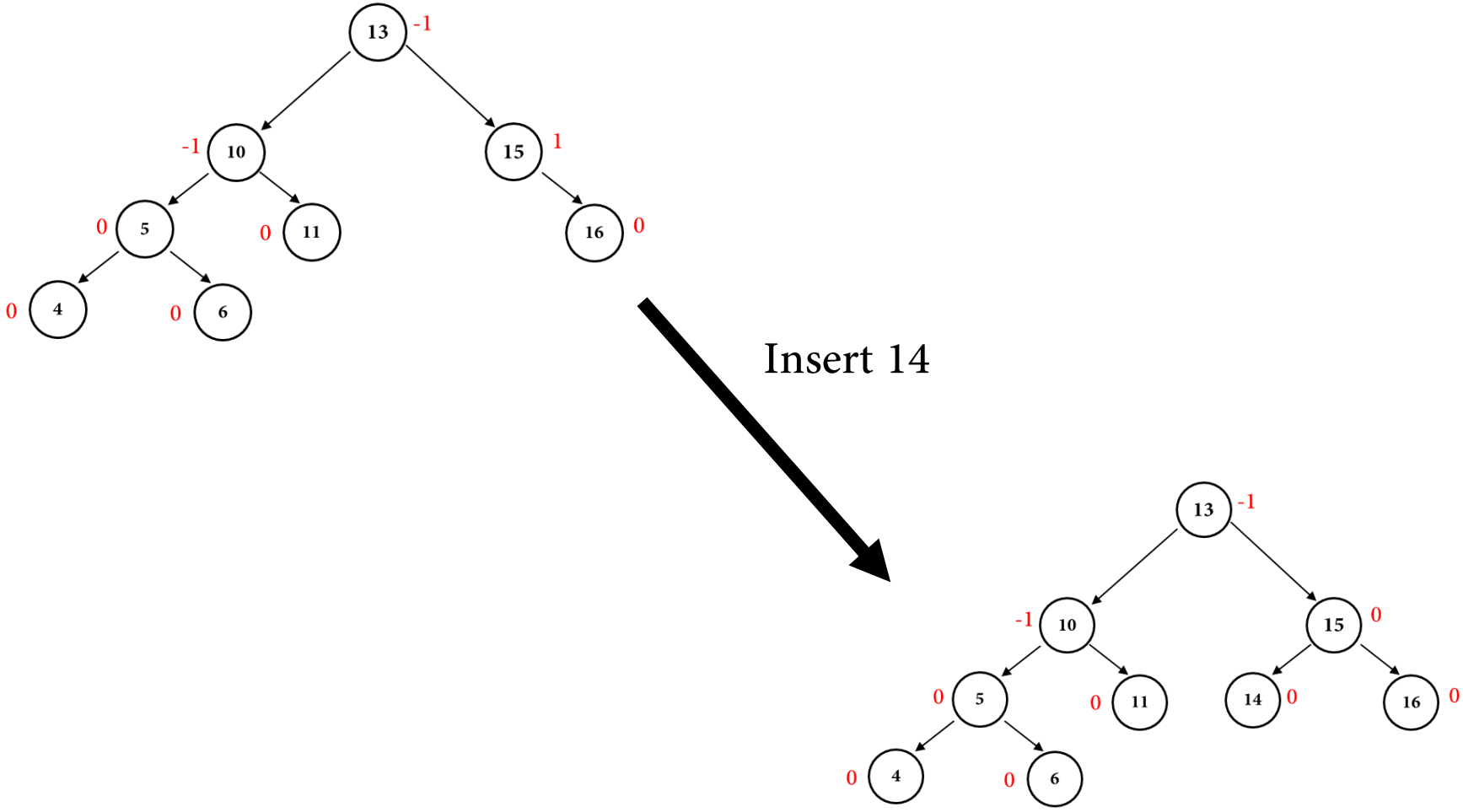
- Only nodes on the path from insertion point to root have possibly changed in height
- Rebalancing is done at the deepest or lowest unbalanced ancestor of the inserted node

- **Three AVL Insertion Cases**
  - **No imbalance:** No rotation needed.
  - **Same-side imbalance (LL/RR):** Requires **single rotation**.
  - **Opposite-side imbalance (LR/RL):** Requires **double rotation**.



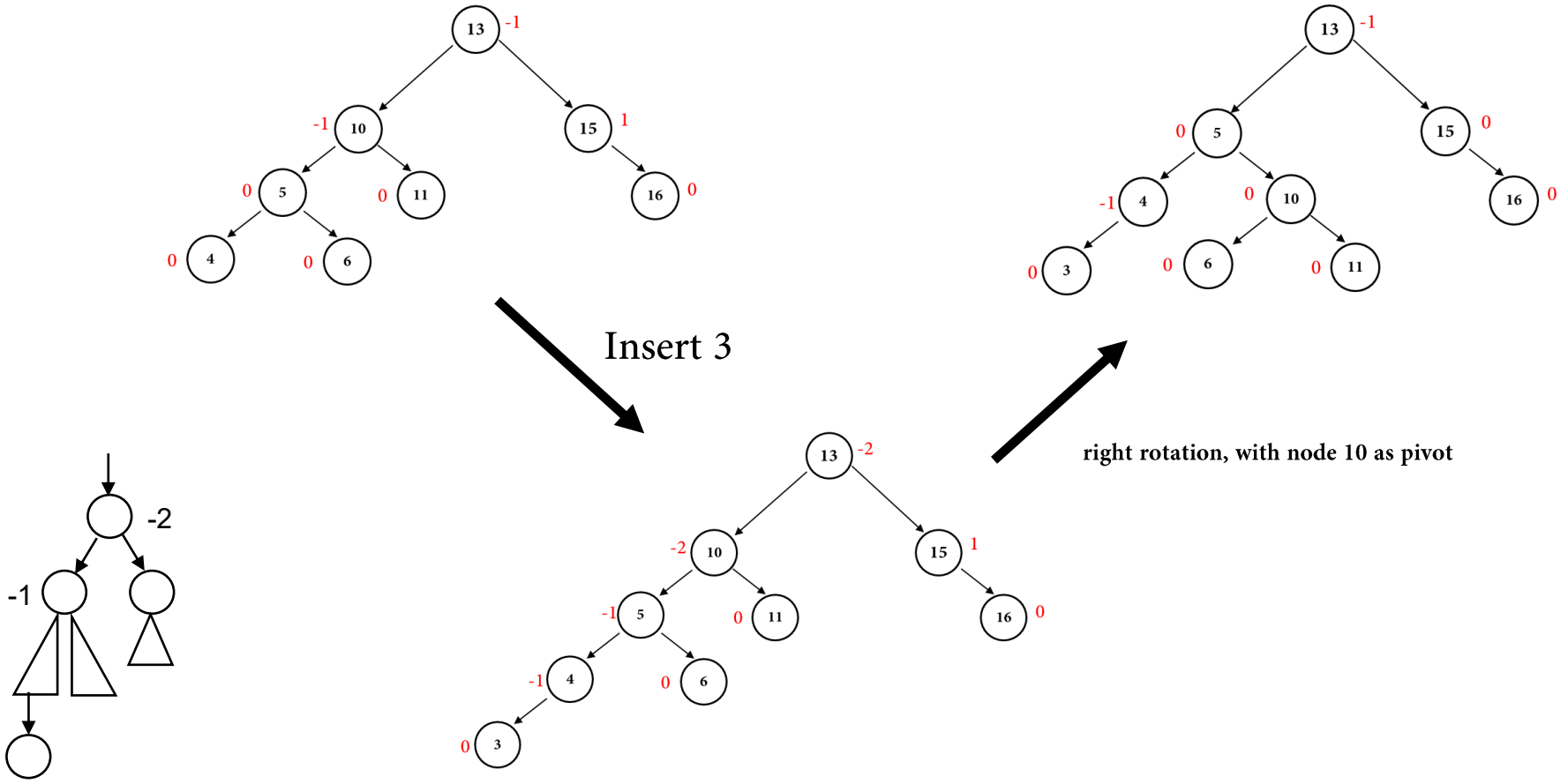
Insertion: case 1

- Example: An insertion that does not cause an imbalance.

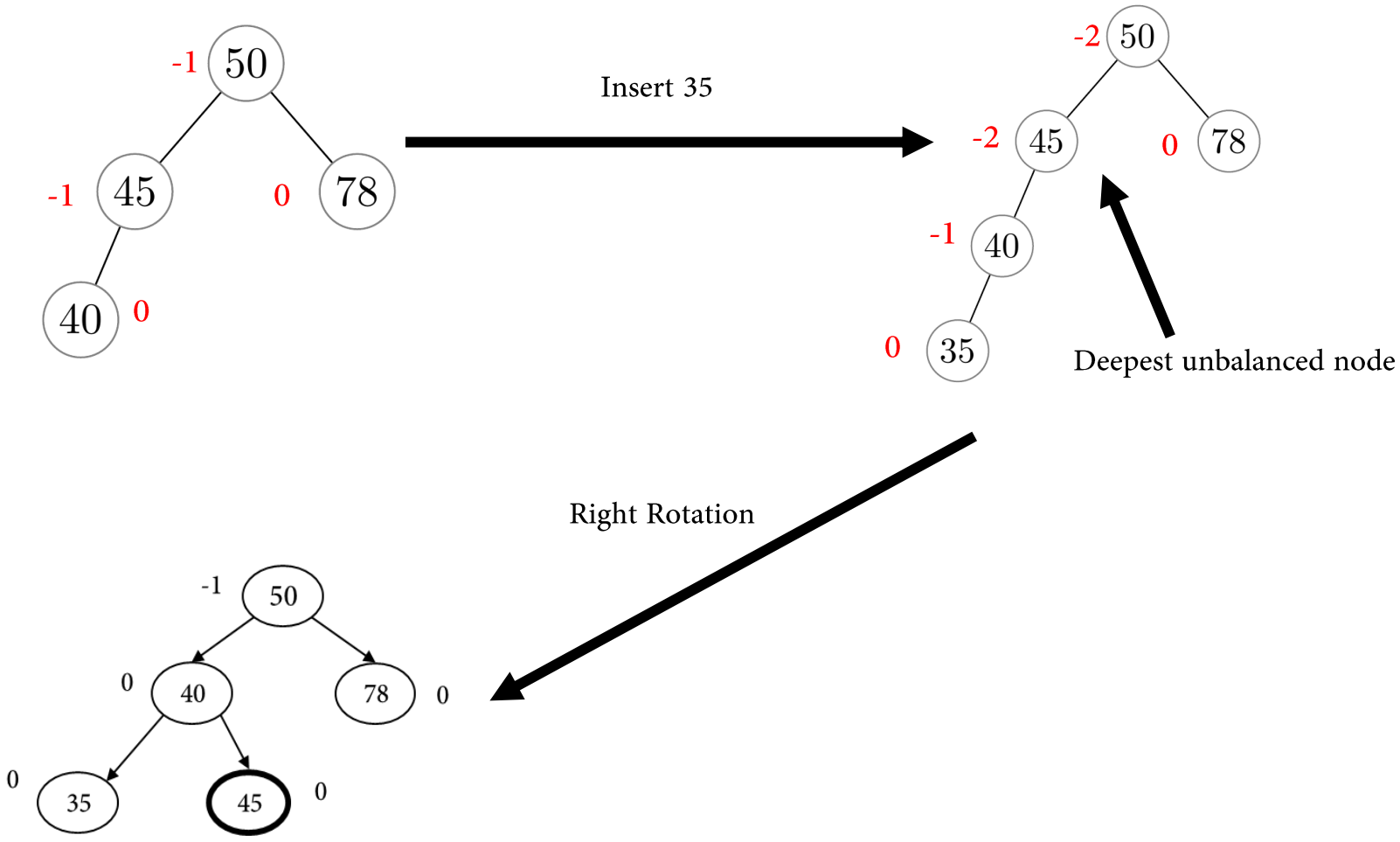


Insertion: case 2

- Case 2a: The lowest node (with a balance factor of -2) had a taller left-subtree and the insertion was on the left-subtree of its left child, LL.
- Requires single right rotation to rebalance.

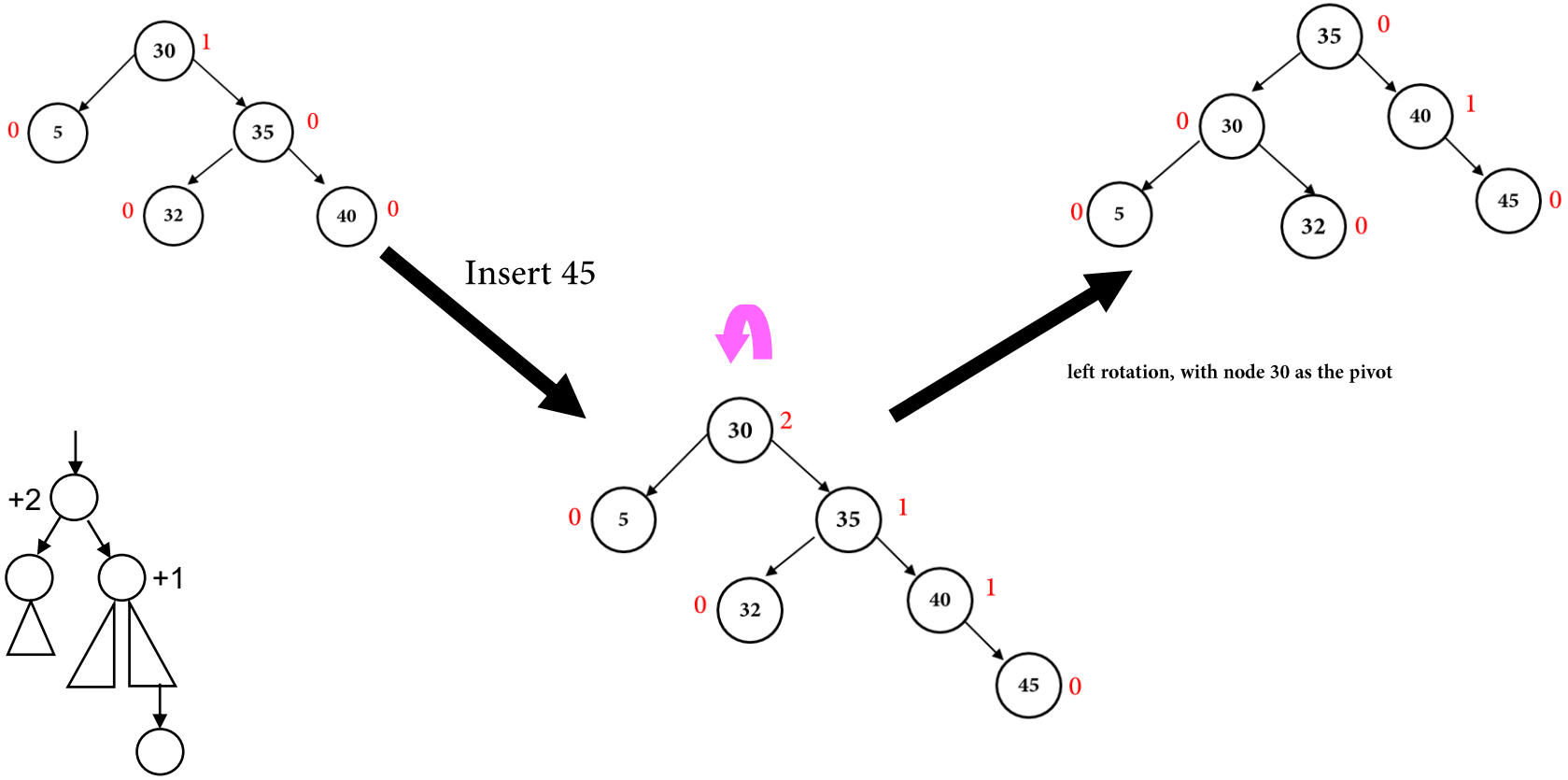


Insertion: case 2...



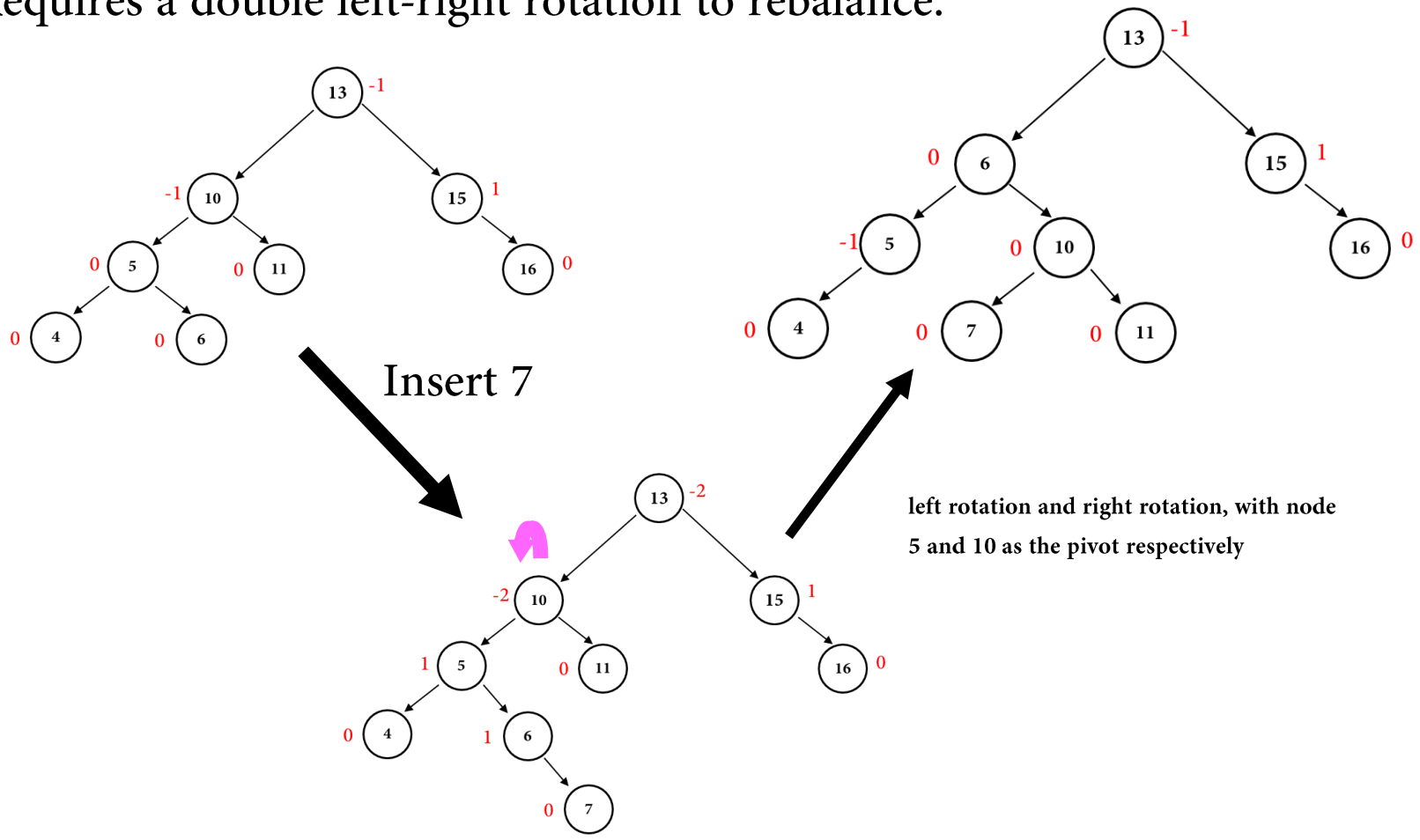
Insertion: case 2...

- Case 2b: The lowest node (with a balance factor of +2) had a taller right-subtree and the insertion was on the right-subtree of its right child, RR.
- Requires single left rotation to rebalance.



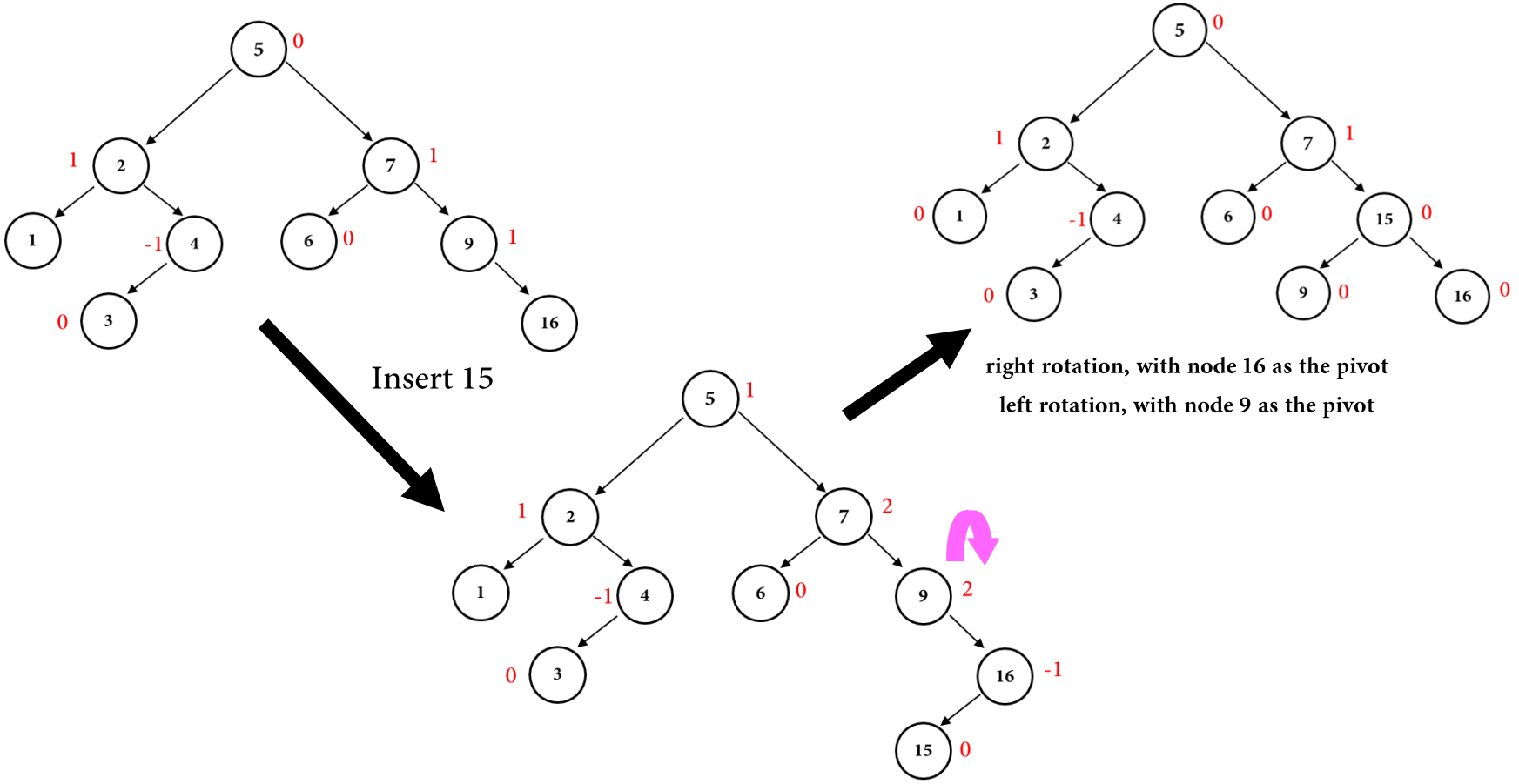
Insertion: case 3

- Case 3a: The lowest node (with a balance factor of -2) had a taller left-subtree and the insertion was on the right-subtree of its left child, LR.
- Requires a double left-right rotation to rebalance.



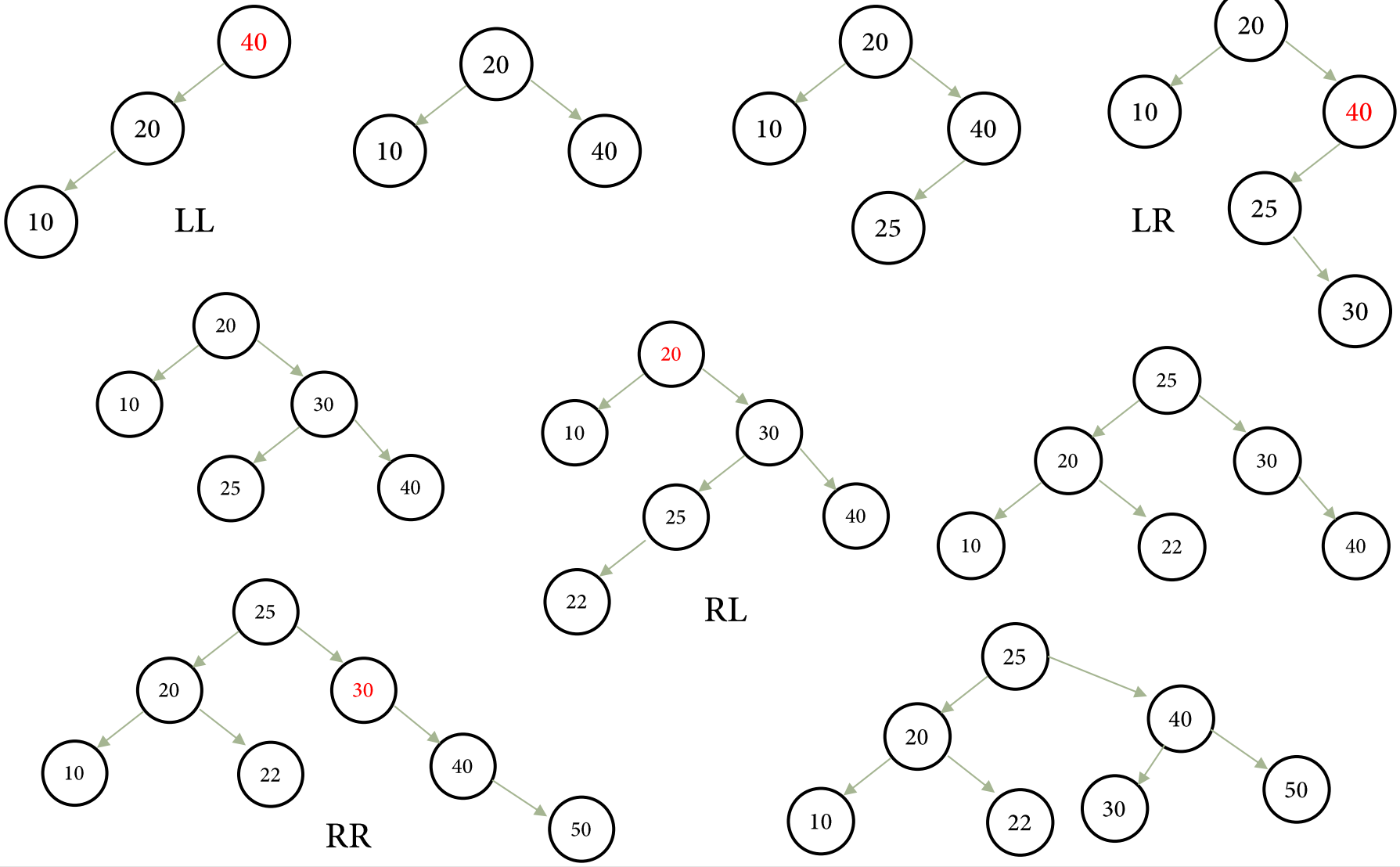
Insertion: case 3 (contd)

- Case 3b: The lowest node (with a balance factor of +2) had a taller right-subtree and the insertion was on the left-subtree of its right child.
- Requires a double right-left rotation to rebalance.



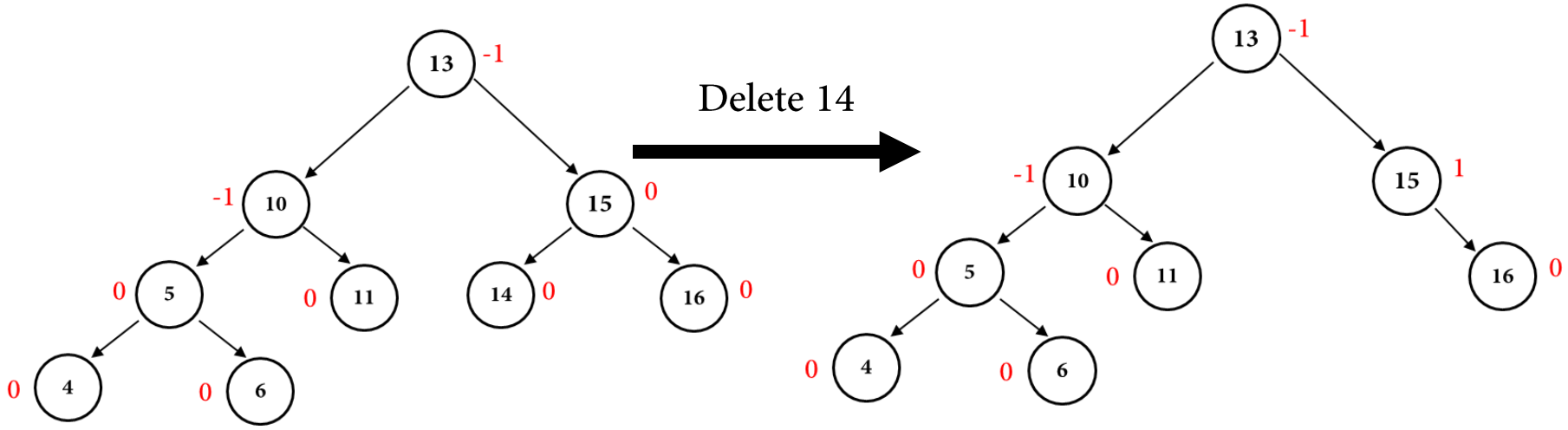
AVL Insert Example

- KEYS: 40, 20, 10, 25, 30, 22, 50

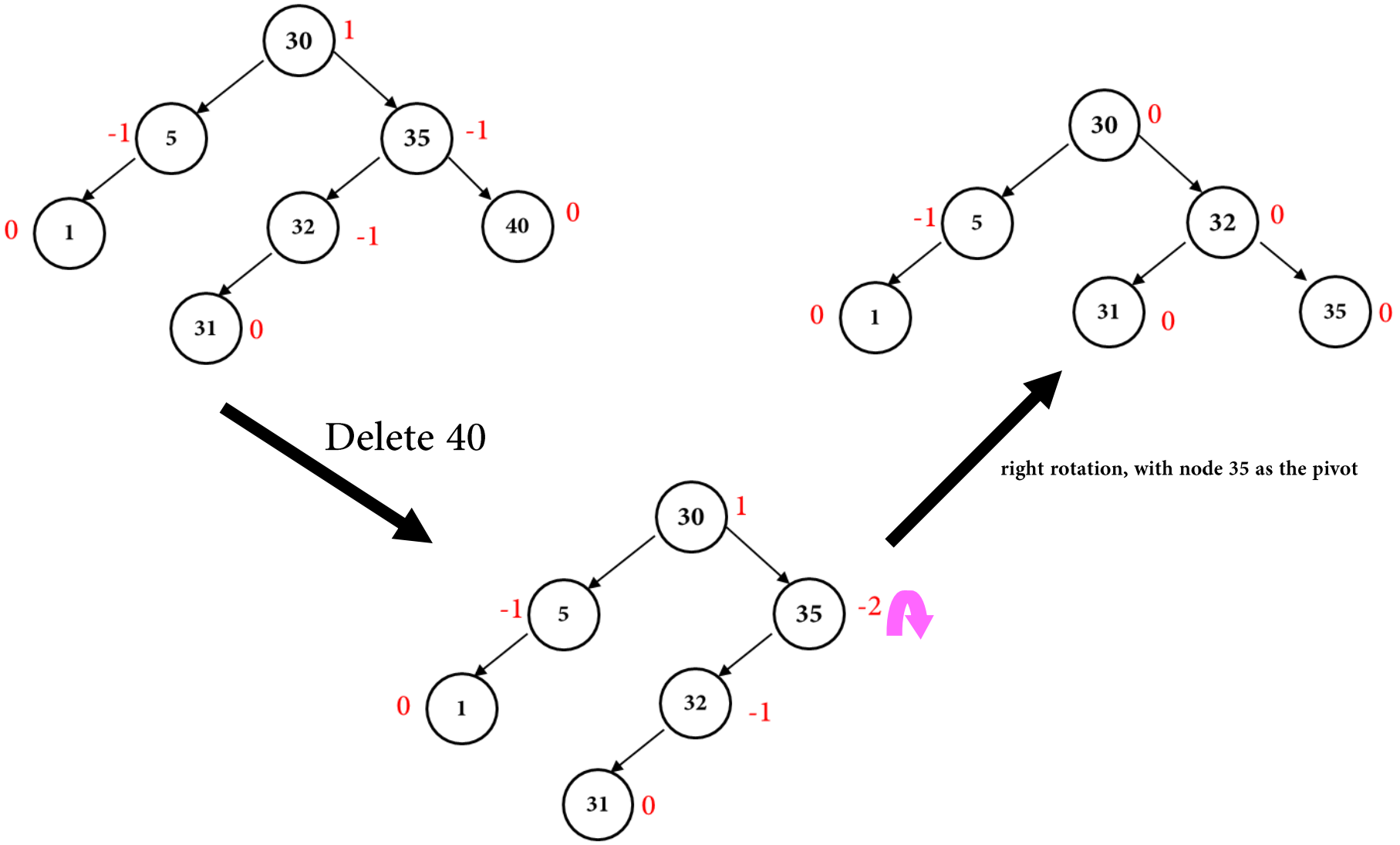


Deletion

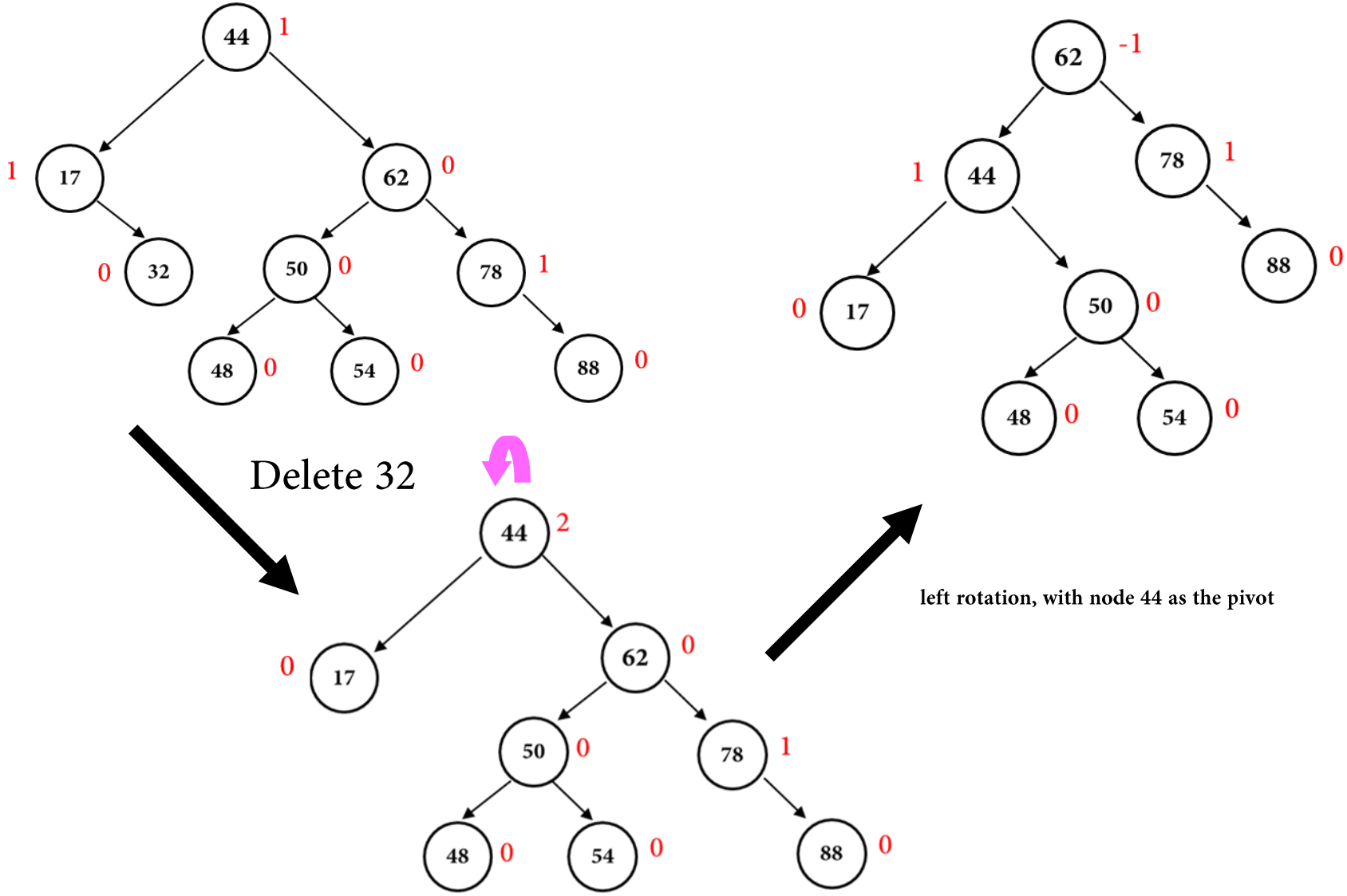
- Delete by a BST deletion by copying algorithm.
- Rebalance the tree if an imbalance occurs.
- There are three deletion cases:
  - Deletion that does not cause an imbalance.
  - Deletion that requires a single rotation to rebalance.
  - Deletion that requires two or more rotations to rebalance.
- Deletion case 1 example:



Deletion: case 2 examples



Deletion: case 2 examples…



Deletion: case 3 examples

